

Patrones de arquitectura en Android con Jetpack Compose

Material de clase

Materia: Desarrollo de Aplicaciones I - Lenguajes Visuales

Enfoque: Android nativo, Kotlin, Jetpack Compose y arquitectura de software

Resumen

Este apunte presenta los principales patrones utilizados en el desarrollo moderno de aplicaciones Android con Jetpack Compose. El objetivo no es memorizar nombres de patrones, sino comprender por qué aparecen, qué problema resuelven, cómo se relacionan entre sí y qué errores arquitectónicos ayudan a evitar. Se abordan patrones de UI declarativa, manejo de estado, flujo unidireccional de datos, ViewModel, Repository, casos de uso, inyección de dependencias, efectos secundarios, navegación y estrategias de escalabilidad.

Idea rectora del apunte

Jetpack Compose no es solamente una forma nueva de dibujar interfaces. Es un cambio de modelo mental: la pantalla deja de ser un conjunto de objetos visuales modificados manualmente y pasa a ser una representación del estado de la aplicación.

Índice

1. Introducción	4
2. De la UI imperativa a la UI declarativa	4
3. Patrón de composición de UI	5
3.1. Ventajas	5
4. Patrón Screen - Content	6
5. Stateful y stateless composables	7
5.1. Composable stateful	7
5.2. Composable stateless	7
6. State hoisting	8
6.1. Fundamento técnico	8
7. Unidirectional Data Flow	9
7.1. Comparación	9
8. UI State Pattern	10
8.1. Estados de carga, éxito y error	10
9. ViewModel Pattern	11
9.1. Responsabilidades del ViewModel	11
10. ViewModel con StateFlow	12
11. Eventos de UI y acciones de usuario	12
12. One-shot events	13
13. Side Effects Pattern	14
13.1. LaunchedEffect	14
13.2. DisposableEffect	15
13.3. rememberCoroutineScope	15
14. Repository Pattern	16
15. DataSource Pattern	16
16. DTO, Entity y Domain Model	17
17. Use Case Pattern	18
18. Clean Architecture adaptada a Android	19
18.1. Capas	19
19. Dependency Injection Pattern	20

20.Navigation Pattern en Compose	21
20.1. Regla de separación	21
21.Single Source of Truth	22
22.Offline First Pattern	22
23.Patrones de rendimiento en Compose	23
23.1. Evitar trabajo costoso en composables	23
23.2. derivedStateOf	24
23.3. Keys en listas	24
24.Patrones de pruebas	24
24.1. Testear Use Cases	24
24.2. Testear ViewModel	25
24.3. Testear Composables stateless	25
25.Anti-patrones frecuentes	25
25.1. God Composable	25
25.2. ViewModel obeso	25
25.3. Repository anémico	26
25.4. Estado duplicado	26
25.5. Efectos secundarios sin control	26
26.Tabla integradora de patrones	27
27.Arquitectura recomendada para proyectos iniciales	27
28.A modo de cierre	28
29.Referencias sugeridas	28

1 Introducción

Cuando se enseña Android con Jetpack Compose, es frecuente comenzar por elementos visibles: Text, Button, Column, Row, TextField o Scaffold. Sin embargo, una aplicación real no se sostiene solamente con componentes visuales. A medida que el sistema crece aparecen problemas de estado, navegación, validación, persistencia, asincronismo, errores de red, sincronización, seguridad, pruebas y mantenimiento.

En ese contexto, los patrones arquitectónicos permiten ordenar el diseño de la aplicación. No son adornos teóricos; son respuestas a problemas concretos de ingeniería.

Problema central

Una app Android moderna debe responder a eventos del usuario, consultar datos, mantener estado, persistir información, navegar entre pantallas, manejar errores y recomponer la UI sin volverse impredecible. Los patrones permiten que esas responsabilidades no queden mezcladas en una misma función o clase.

En Compose, esta necesidad se vuelve todavía más evidente porque los composables pueden recomponerse muchas veces. Por lo tanto, una función composable no debería contener lógica pesada, efectos secundarios arbitrarios ni acceso directo a fuentes de datos. Su responsabilidad principal debería ser describir la interfaz a partir de un estado.

2 De la UI imperativa a la UI declarativa

El primer patrón conceptual de Compose es la UI declarativa. En el modelo clásico de Android, muchas pantallas se construían declarando vistas en XML y luego modificándolas desde código Kotlin o Java.

```
val texto = findViewById<TextView>(R.id.texto)
texto.text = "Resultado actualizado"
```

Ese modelo es imperativo porque el programador indica paso a paso qué objeto modificar y cuándo hacerlo. En Compose, en cambio, se declara qué debe mostrarse para un estado dado.

```
@Composable
fun Saludo(nombre: String) {
    Text(text = "Hola, $nombre")
}
```

Si cambia nombre, Compose puede volver a ejecutar el composable y actualizar la interfaz correspondiente.

Fundamento técnico

Compose funciona sobre un modelo de composición y recomposición. Una función composable puede ejecutarse varias veces durante la vida de una pantalla. Por eso, el cuerpo de un composable debe ser lo más puro y predecible posible: recibe estado, describe UI y delega eventos.

Desde el punto de vista de ingeniería, el cambio importante es este:

UI imperativa:

buscar vista -> modificar propiedad -> sincronizar manualmente

UI declarativa:

estado -> función composible -> descripción de la UI

3 Patrón de composición de UI

Compose promueve construir interfaces mediante funciones pequeñas que se combinan entre sí. Este patrón se puede llamar composición de UI.

```
@Composable
fun PantallaPrincipal() {
    Column {
        Encabezado()
        Formulario()
        Resultado()
    }
}
```

Cada componente representa una parte de la pantalla. La pantalla completa se obtiene por composición de componentes menores.

3.1 Ventajas

- Mejora la legibilidad.
- Permite reutilizar partes de la UI.
- Reduce composables gigantes.
- Facilita pruebas visuales y unitarias.
- Permite separar componentes genéricos de pantallas concretas.

Error común

Un error típico en estudiantes es construir toda la pantalla dentro de una única función `@Composable`. Cuando la función crece demasiado, se vuelve difícil saber qué parte maneja estado, qué parte dibuja UI y qué parte responde a eventos.

Ejemplo de descomposición:

```
@Composable
fun PantallaLogin(
    estado: LoginUiState,
    onUsuarioChange: (String) -> Unit,
    onPasswordChange: (String) -> Unit,
    onIngresarClick: () -> Unit
```

```
) {  
    Column {  
        TituloLogin()  
        CamposLogin(  
            usuario = estado.usuario,  
            password = estado.password,  
            onUsuarioChange = onUsuarioChange,  
            onPasswordChange = onPasswordChange  
        )  
        BotonIngresar(  
            habilitado = estado.puedeIngresar,  
            onClick = onIngresarClick  
        )  
    }  
}
```

4 Patrón Screen - Content

Un patrón práctico muy usado en Compose es separar el composible de pantalla del composible de contenido.

```
@Composable  
fun LoginScreen(viewModel: LoginViewModel) {  
    val estado = viewModel.estado  
  
    LoginContent(  
        estado = estado,  
        onUsuarioChange = viewModel::onUsuarioChange,  
        onPasswordChange = viewModel::onPasswordChange,  
        onIngresarClick = viewModel::ingresar  
    )  
}
```

El composible LoginScreen conecta la UI con el ViewModel. El composible LoginContent solamente renderiza datos y expone eventos.

```
@Composable  
fun LoginContent(  
    estado: LoginUiState,  
    onUsuarioChange: (String) -> Unit,  
    onPasswordChange: (String) -> Unit,  
    onIngresarClick: () -> Unit  
) {  
    Column {  
        TextField(  
            value = estado.usuario,  
            onValueChange = onUsuarioChange,  
            label = { Text("Usuario") }  
        )  
    }  
}
```

```
    )

    TextField(
        value = estado.password,
        onValueChange = onPasswordChange,
        label = { Text("Contraseña") }
    )

    Button(
        onClick = onIngresarClick,
        enabled = estado.puedeIngresar
    ) {
        Text("Ingresar")
    }
}
}
```

Ventaja pedagógica

Este patrón ayuda a distinguir entre un composable conectado a la arquitectura y un composable reutilizable, testeable y visualmente predecible.

5 Stateful y stateless composables

Un composable *stateful* guarda o administra estado internamente. Un composable *stateless* recibe el estado desde afuera y emite eventos hacia afuera.

5.1 Composable stateful

```
@Composable
fun CampoNombreStateful() {
    var nombre by remember { mutableStateOf("") }

    TextField(
        value = nombre,
        onValueChange = { nombre = it }
    )
}
```

Este composable funciona, pero está acoplado a su propio estado interno.

5.2 Composable stateless

```
@Composable
fun CampoNombre(
    nombre: String,
```

```
        onNombreChange: (String) -> Unit
    ) {
        TextField(
            value = nombre,
            onValueChange = onNombreChange
        )
    }
```

Ahora el composable no decide dónde vive el estado. Solamente lo muestra y notifica cambios.

Regla práctica

Cuanto más reutilizable debe ser un composable, más conveniente es que sea stateless. El estado debería vivir en el nivel más bajo que permita compartirlo correctamente y en el nivel más alto que permita controlarlo de manera coherente.

6 State hoisting

El *state hoisting* consiste en elevar el estado desde un composable hijo hacia su llamador. En lugar de que el hijo guarde su estado, recibe un valor y una función de cambio.

```
@Composable
fun CampoTexto(
    valor: String,
    onValorChange: (String) -> Unit
) {
    TextField(
        value = valor,
        onValueChange = onValorChange
    )
}
```

Uso desde el padre:

```
@Composable
fun Formulario() {
    var texto by remember { mutableStateOf("") }

    CampoTexto(
        valor = texto,
        onValorChange = { texto = it }
    )
}
```

6.1 Fundamento técnico

El patrón reemplaza una variable interna por dos parámetros:


```
value: T  
onValueChange: (T) -> Unit
```

Esto permite que el estado sea controlado por un componente superior, por un ViewModel o por cualquier state holder.

Beneficio

El state hoisting permite desacoplar la representación visual de la política de estado. El composable no necesita saber si el dato viene de memoria, de un ViewModel, de una base local o de una respuesta de backend.

7 Unidirectional Data Flow

El flujo unidireccional de datos, o UDF, es uno de los patrones centrales en Compose.

Estado baja hacia la UI
Eventos suben hacia quien modifica el estado

State -> UI -> Event -> State holder -> State

La UI no modifica directamente el estado de negocio. En lugar de eso, emite eventos. El responsable del estado procesa esos eventos y genera un nuevo estado.

```
@Composable  
fun PerfilScreen(viewModel: PerfilViewModel) {  
    val estado = viewModel.estado  
  
    PerfilContent(  
        estado = estado,  
        onNombreChange = { viewModel.onNombreChange(it) },  
        onGuardarClick = { viewModel.guardar() }  
    )  
}
```

Fundamento técnico

UDF reduce la cantidad de caminos posibles por los que puede cambiar el estado. Esto mejora la predictibilidad, facilita el debugging y evita que la UI quede en estados inconsistentes.

7.1 Comparación

Modelo	Consecuencia
Flujo bidireccional desordenado	Muchos componentes pueden cambiar el estado, dificultando rastrear errores.
Flujo unidireccional	Los eventos tienen una dirección clara y el estado se produce en un punto controlado.

8 UI State Pattern

Una buena práctica en Compose es representar el estado completo de una pantalla mediante una clase o estructura específica.

```
data class LoginUiState(  
    val usuario: String = "",  
    val password: String = "",  
    val cargando: Boolean = false,  
    val error: String? = null  
)
```

Esto evita manejar muchas variables sueltas.

```
var usuario by mutableStateOf("")  
var password by mutableStateOf("")  
var cargando by mutableStateOf(false)  
var error by mutableStateOf<String?>(null)
```

El estado agrupado es más expresivo:

```
val estado = LoginUiState(  
    usuario = "ana",  
    password = "",  
    cargando = false,  
    error = null  
)
```

Criterio de diseño

Si una pantalla tiene varias variables relacionadas entre sí, probablemente conviene modelarlas como un único objeto de estado.

8.1 Estados de carga, éxito y error

Muchas pantallas remotas necesitan representar carga, éxito y error.

```
sealed interface UsuariosUiState {  
    object Loading : UsuariosUiState  
    data class Success(val usuarios: List<Usuario>) : UsuariosUiState  
    data class Error(val mensaje: String) : UsuariosUiState  
}
```

Uso en UI:

```
@Composable  
fun UsuariosContent(estado: UsuariosUiState) {  
    when (estado) {  
        is UsuariosUiState.Loading -> Text("Cargando...")  
        is UsuariosUiState.Success -> ListaUsuarios(estado.usuarios)  
        is UsuariosUiState.Error -> Text(estado.mensaje)  
    }  
}
```

```
    }  
}
```

Este patrón evita estados inválidos. Por ejemplo, evita tener simultáneamente cargando = true, datos cargados y error activo.

9 ViewModel Pattern

El ViewModel es el state holder más habitual para pantallas Android. Su función no es dibujar UI, sino conservar y producir estado de UI, responder eventos y coordinar la interacción con la capa de dominio o datos.

```
class LoginViewModel(  
    private val repository: UsuarioRepository  
) : ViewModel() {  
  
    var estado by mutableStateOf(LoginUiState())  
    private set  
  
    fun onUsuarioChange(usuario: String) {  
        estado = estado.copy(usuario = usuario)  
    }  
  
    fun onPasswordChange(password: String) {  
        estado = estado.copy(password = password)  
    }  
  
    fun ingresar() {  
        // Validar y delegar en repository o use case  
    }  
}
```

9.1 Responsabilidades del ViewModel

- Mantener estado de UI.
- Procesar eventos de la pantalla.
- Lanzar operaciones asincrónicas con viewModelScope.
- Invocar casos de uso o repositorios.
- Exponer estado observable a Compose.
- Evitar que la UI conozca detalles de persistencia, red o reglas de negocio.

Límite importante

El ViewModel no debería contener lógica de infraestructura, SQL, llamadas HTTP crudas ni manipulación directa de archivos. Eso pertenece a la capa de datos.

10 ViewModel con StateFlow

En aplicaciones modernas es común exponer estado desde el ViewModel usando StateFlow.

```
class UsuariosViewModel(  
    private val repository: UsuarioRepository  
) : ViewModel() {  
  
    private val _estado =  
        MutableStateFlow<UsuariosUiState>(UsuariosUiState.Loading)  
  
    val estado: StateFlow<UsuariosUiState> = _estado  
  
    init {  
        cargarUsuarios()  
    }  
  
    private fun cargarUsuarios() {  
        viewModelScope.launch {  
            try {  
                val usuarios = repository.obtenerUsuarios()  
                _estado.value = UsuariosUiState.Success(usuarios)  
            } catch (e: Exception) {  
                _estado.value = UsuariosUiState.Error("No se pudo cargar")  
            }  
        }  
    }  
}
```

En Compose:

```
@Composable  
fun UsuariosScreen(viewModel: UsuariosViewModel) {  
    val estado by viewModel.estado.collectAsState()  
  
    UsuariosContent(estado = estado)  
}
```

Fundamento técnico

StateFlow representa un flujo observable con un valor actual. Esto encaja bien con Compose porque la UI puede observar ese valor y recomponerse cuando cambia.

11 Eventos de UI y acciones de usuario

Una pantalla no solo muestra estado: también recibe acciones. Una estrategia robusta es modelar esas acciones como eventos.

```
sealed interface LoginEvent {
```

```
data class UsuarioChanged(val value: String) : LoginEvent
data class PasswordChanged(val value: String) : LoginEvent
object Submit : LoginEvent
}
```

El ViewModel expone una función única:

```
fun onEvent(event: LoginEvent) {
    when (event) {
        is LoginEvent.UsuarioChanged -> {
            estado = estado.copy(usuario = event.value)
        }

        is LoginEvent.PasswordChanged -> {
            estado = estado.copy(password = event.value)
        }

        LoginEvent.Submit -> {
            ingresar()
        }
    }
}
```

Uso desde Compose:

```
LoginContent(
    estado = estado,
    onEvent = viewModel::onEvent
)
```

Ventaja

El patrón de eventos centraliza la intención del usuario y evita tener muchas funciones públicas dispersas en el ViewModel.

12 One-shot events

No todo es estado persistente. Algunas acciones ocurren una sola vez: mostrar un Snackbar, navegar, cerrar sesión, pedir permiso o mostrar un mensaje temporal.

No conviene modelar estas acciones como estado permanente de pantalla si eso puede provocar que se repitan luego de una recomposición.

```
sealed interface LoginEffect {
    object NavegarHome : LoginEffect
    data class MostrarMensaje(val texto: String) : LoginEffect
}
```

Una forma común de emitirlos es con SharedFlow.

```
class LoginViewModel : ViewModel() {
```

```
private val _effects = MutableSharedFlow<LoginEffect>()
val effects = _effects.asSharedFlow()

fun ingresar() {
    viewModelScope.launch {
        _effects.emit(LoginEffect.NavegarHome)
    }
}
```

En Compose:

```
@Composable
fun LoginScreen(viewModel: LoginViewModel) {
    LaunchedEffect(Unit) {
        viewModel.effects.collect { effect ->
            when (effect) {
                LoginEffect.NavegarHome -> {
                    // navegar
                }

                is LoginEffect.MostrarMensaje -> {
                    // mostrar snackbar
                }
            }
        }
    }
}
```

Cuidado

Un error frecuente es guardar eventos únicos como un campo del estado, por ejemplo `navegar = true`. Si no se consume correctamente, puede repetirse al recomponer o al recrear la pantalla.

13 Side Effects Pattern

En Compose, un side effect es una operación que ocurre fuera de la composición pura. Por ejemplo: iniciar una corrutina, mostrar un snackbar, registrar un listener, navegar o interactuar con APIs externas.

13.1 LaunchedEffect

Sirve para ejecutar una corrutina asociada al ciclo de vida del composable.

```
@Composable
fun PantallaDetalle(id: Int, viewModel: DetalleViewModel) {
```

```
    LaunchedEffect(id) {  
        viewModel.cargarDetalle(id)  
    }  
  
    DetalleContent(estado = viewModel.estado)  
}
```

13.2 DisposableEffect

Sirve para registrar y liberar recursos.

```
@Composable  
fun SensorScreen(sensorManager: SensorManager) {  
    DisposableEffect(Unit) {  
        val listener = crearListener()  
        sensorManager.registerListener(listener)  
  
        onDispose {  
            sensorManager.unregisterListener(listener)  
        }  
    }  
}
```

13.3 rememberCoroutineScope

Permite lanzar una corrutina en respuesta a un evento de UI.

```
@Composable  
fun PantallaConSnackbar(snackbarHostState: SnackbarHostState) {  
    val scope = rememberCoroutineScope()  
  
    Button(  
        onClick = {  
            scope.launch {  
                snackbarHostState.showSnackbar("Guardado")  
            }  
        }  
    ) {  
        Text("Guardar")  
    }  
}
```

Riesgo

Los side effects son necesarios, pero abusar de ellos rompe el modelo declarativo. Deben usarse para efectos relacionados con la UI y de manera controlada.

14 Repository Pattern

El patrón Repository abstrae el origen de los datos. La UI y el ViewModel no deberían saber si los datos vienen de Room, Retrofit, DataStore, memoria, archivos o backend.

ViewModel -> Repository -> DataSource

Contrato:

```
interface UsuarioRepository {  
    suspend fun obtenerUsuarios(): List<Usuario>  
    suspend fun guardarUsuario(usuario: Usuario)  
}
```

Implementación:

```
class UsuarioRepositoryImpl(  
    private val localDataSource: UsuarioLocalDataSource,  
    private val remoteDataSource: UsuarioRemoteDataSource  
) : UsuarioRepository {  
  
    override suspend fun obtenerUsuarios(): List<Usuario> {  
        return localDataSource.obtenerUsuarios()  
    }  
  
    override suspend fun guardarUsuario(usuario: Usuario) {  
        remoteDataSource.guardarUsuario(usuario)  
        localDataSource.guardarUsuario(usuario)  
    }  
}
```

Fundamento técnico

El Repository desacopla la lógica de presentación del mecanismo concreto de acceso a datos. Esto permite cambiar Room por una API, agregar cache, implementar sincronización o crear repositorios falsos para pruebas.

15 DataSource Pattern

Un DataSource representa una fuente concreta de datos.

```
UsuarioLocalDataSource -> Room  
UsuarioRemoteDataSource -> Retrofit  
UsuarioPreferencesDataSource -> DataStore
```

Ejemplo:

```
class UsuarioLocalDataSource(  
    private val usuarioDao: UsuarioDao  
) {  
    suspend fun obtenerUsuarios(): List<UsuarioEntity> {
```



```
        return usuarioDao.obtenerTodos()
    }
}

class UsuarioRemoteDataSource(
    private val api: UsuarioApi
) {
    suspend fun obtenerUsuarios(): List<UsuarioDto> {
        return api.obtenerUsuarios()
    }
}
```

El Repository puede combinar ambos.

Diferencia clave

El DataSource sabe cómo hablar con una fuente concreta. El Repository decide cómo combinar fuentes, resolver conflictos, transformar datos y exponer una API coherente al resto de la app.

16 DTO, Entity y Domain Model

En una arquitectura limpia, no conviene usar la misma clase para todo. Es preferible distinguir entre modelos remotos, modelos locales y modelos de dominio.

DTO -> modelo usado por la API
Entity -> modelo usado por Room
Domain -> modelo usado por la lógica de la app

Ejemplo:

```
data class UsuarioDto(
    val id: Int,
    val fullName: String,
    val email: String
)

@Entity(tableName = "usuarios")
data class UsuarioEntity(
    @PrimaryKey val id: Int,
    val nombre: String,
    val email: String
)

data class Usuario(
    val id: Int,
    val nombre: String,
    val email: String
)
```

Mappers:

```
fun UsuarioDto.toDomain(): Usuario {
    return Usuario(
        id = id,
        nombre = fullName,
        email = email
    )
}

fun UsuarioEntity.toDomain(): Usuario {
    return Usuario(
        id = id,
        nombre = nombre,
        email = email
    )
}
```

Error común

Exponer entidades de Room directamente en la UI acopla toda la aplicación al esquema de base de datos. Si cambia la tabla, cambia la UI. Eso viola separación de responsabilidades.

17 Use Case Pattern

Los casos de uso representan operaciones de negocio o de aplicación. Se ubican entre el ViewModel y el Repository.

ViewModel -> UseCase -> Repository

Ejemplo:

```
class LoginUseCase(
    private val repository: UsuarioRepository
) {
    suspend operator fun invoke(
        usuario: String,
        password: String
    ): LoginResult {
        if (usuario.isBlank()) {
            return LoginResult.Error("Usuario requerido")
        }

        if (password.length < 8) {
            return LoginResult.Error("Contraseña inválida")
        }

        return repository.login(usuario, password)
    }
}
```

Uso desde ViewModel:

```
class LoginViewModel(  
    private val loginUseCase: LoginUseCase  
) : ViewModel() {  
  
    fun ingresar() {  
        viewModelScope.launch {  
            val result = loginUseCase(  
                estado.usuario,  
                estado.password  
            )  
        }  
    }  
}
```

Fundamento

El Use Case evita que el ViewModel se transforme en una clase con demasiada lógica. También permite testear reglas de aplicación sin depender de Compose ni del ciclo de vida Android.

18 Clean Architecture adaptada a Android

Una estructura habitual en Android moderno es una adaptación de Clean Architecture:

UI -> ViewModel -> UseCase -> Repository -> DataSource

18.1 Capas

Capa	Responsabilidad
UI	Renderizar estado y emitir eventos.
ViewModel	Mantener estado de UI y coordinar casos de uso.
Domain	Contener modelos puros y reglas de negocio.
Data	Implementar repositorios, DAOs, APIs, DataStore y mappers.

Regla de dependencia

Las capas internas no deberían depender de detalles externos. El dominio no debería conocer Compose, Room, Retrofit ni Android Context.

Ejemplo de organización:

```
com.ejemplo.app/  
    ui/  
        login/  
            LoginScreen.kt  
            LoginContent.kt
```

```
        LoginViewModel.kt
        LoginUiState.kt
domain/
  model/
    Usuario.kt
  usecase/
    LoginUseCase.kt
data/
  repository/
    UsuarioRepositoryImpl.kt
  local/
    UsuarioDao.kt
    UsuarioEntity.kt
  remote/
    UsuarioApi.kt
    UsuarioDto.kt
  mapper/
    UsuarioMapper.kt
```

19 Dependency Injection Pattern

La inyección de dependencias consiste en entregar a una clase los objetos que necesita, en lugar de crearlos internamente. En Android se usa frecuentemente Hilt.

Sin inyección:

```
class LoginViewModel : ViewModel() {
    private val repository = UsuarioRepositoryImpl()
}
```

Problema: el ViewModel queda acoplado a una implementación concreta.

Con inyección:

```
class LoginViewModel(
    private val repository: UsuarioRepository
) : ViewModel()
```

Ahora se puede inyectar una implementación real o una falsa para pruebas.

Ejemplo conceptual con Hilt:

```
@HiltViewModel
class LoginViewModel @Inject constructor(
    private val loginUseCase: LoginUseCase
) : ViewModel()
```

Módulo:

```
@Module
@InstallIn(SingletonComponent::class)
```

```
object RepositoryModule {  
  
    @Provides  
    fun provideUsuarioRepository(  
        api: UsuarioApi,  
        dao: UsuarioDao  
    ): UsuarioRepository {  
        return UsuarioRepositoryImpl(api, dao)  
    }  
}
```

Ventaja de ingeniería

La inyección de dependencias reduce acoplamiento, facilita pruebas, centraliza la construcción de objetos y evita que las clases de alto nivel conozcan detalles de infraestructura.

20 Navigation Pattern en Compose

La navegación en Compose suele organizarse declarando rutas y destinos composables.

```
@Composable  
fun AppNavigation() {  
    val navController = rememberNavController()  
  
    NavHost(  
        navController = navController,  
        startDestination = "login"  
    ) {  
        composable("login") {  
            LoginScreen(  
                onLoginOk = {  
                    navController.navigate("home")  
                }  
            )  
        }  
  
        composable("home") {  
            HomeScreen()  
        }  
    }  
}
```

20.1 Regla de separación

La pantalla no debería crear rutas complejas ni conocer toda la estructura de navegación global. Una estrategia limpia es que la pantalla emita una intención:

```
LoginScreen(  
    onLoginOk = { navController.navigate("home") }  
)
```

La UI concreta no debería depender directamente de NavController si se quiere mantenerla más testeable.

Error común

Pasar NavController a todos los composables genera acoplamiento. Es preferible pasar callbacks específicos como onBackPressed, onLoginOk u onDetalleClick.

21 Single Source of Truth

Single Source of Truth significa que una parte del sistema debe ser la fuente autorizada de un dato. Esto evita inconsistencias.

Ejemplo incorrecto:

```
La UI tiene una lista de usuarios.  
Room tiene otra lista.  
La API devuelve otra lista.  
El ViewModel mantiene otra copia.
```

Ejemplo más ordenado:

```
Room es fuente local observable.  
La API actualiza Room.  
La UI observa Room.
```

```
API -> Repository -> Room -> ViewModel -> UI
```

Fundamento técnico

Cuando existen múltiples copias independientes del mismo dato, aparecen errores de sincronización. Definir una fuente de verdad reduce inconsistencias y simplifica el razonamiento sobre la app.

22 Offline First Pattern

Offline First significa diseñar la app para funcionar aunque la red no esté disponible. Una estrategia común es que la UI lea siempre desde la base local y que el Repository sincronice con backend.

```
UI observa Room  
Repository sincroniza API -> Room  
Repository envía cambios locales -> API
```

Ejemplo conceptual:

```
class UsuarioOfflineRepository(  
    private val dao: UsuarioDao,  
    private val api: UsuarioApi  
) : UsuarioRepository {  
  
    override fun observarUsuarios(): Flow<List<Usuario>> {  
        return dao.observarUsuarios()  
            .map { lista -> lista.map { it.toDomain() } }  
    }  
  
    suspend fun sincronizar() {  
        val remotos = api.obtenerUsuarios()  
        dao.reemplazarTodos(remotos.map { it.toEntity() })  
    }  
}
```

Ventaja

El usuario puede seguir usando la app sin conexión, y la sincronización se vuelve una responsabilidad explícita del Repository.

23 Patrones de rendimiento en Compose

Compose recompone funciones cuando cambia el estado que leen. Por eso, el diseño del estado afecta directamente al rendimiento.

23.1 Evitar trabajo costoso en composables

Mal:

```
@Composable  
fun ListaUsuarios(usuarios: List<Usuario>) {  
    val ordenados = usuarios.sortedBy { it.nombre }  
  
    LazyColumn {  
        items(ordenados) { usuario ->  
            Text(usuario.nombre)  
        }  
    }  
}
```

Mejor: ordenar antes, por ejemplo en ViewModel o usar memoización si corresponde.

```
val usuariosOrdenados by remember(usuarios) {  
    mutableStateOf(usuarios.sortedBy { it.nombre })  
}
```

23.2 derivedStateOf

Puede usarse cuando un estado derivado cambia menos frecuentemente que sus entradas.

```
val mostrarBoton by remember {  
    derivedStateOf {  
        listaState.firstVisibleItemIndex > 0  
    }  
}
```

23.3 Keys en listas

En listas dinámicas conviene usar claves estables.

```
LazyColumn {  
    items(  
        items = usuarios,  
        key = { usuario -> usuario.id }  
    ) { usuario ->  
        UsuarioItem(usuario)  
    }  
}
```

Cuidado

No todo problema de rendimiento se resuelve con remember. Primero hay que entender qué estado cambia, dónde se lee y qué parte de la composición queda invalidada.

24 Patrones de pruebas

Una arquitectura bien separada facilita pruebas.

24.1 Testear Use Cases

```
@Test  
fun loginConPasswordCortaRetornaError() = runTest {  
    val repository = FakeUsuarioRepository()  
    val useCase = LoginUseCase(repository)  
  
    val result = useCase("ana", "123")  
  
    assertTrue(result is LoginResult.Error)  
}
```


24.2 Testear ViewModel

```
@Test
fun alCambiarUsuarioActualizaEstado() {
    val viewModel = LoginViewModel(FakeLoginUseCase())

    viewModel.onUsuarioChange("ana")

    assertEquals("ana", viewModel.estado.usuario)
}
```

24.3 Testear Composables stateless

Un composable stateless es más fácil de probar porque se le puede pasar un estado fijo y verificar qué muestra.

```
LoginContent(
    estado = LoginUiState(usuario = "ana"),
    onUsuarioChange = {},
    onPasswordChange = {},
    onIngresarClick = {}
)
```

Relación arquitectura-testing

Cuanto menos sabe la UI sobre datos, red, base local y reglas de negocio, más fácil resulta testearla.

25 Anti-patrones frecuentes

25.1 God Composable

Una función composable con cientos de líneas que dibuja todo, valida todo, navega, consulta datos y maneja estado.

Problema

Es difícil de leer, difícil de probar, difícil de mantener y propensa a errores durante recomposición.

25.2 ViewModel obeso

El ViewModel contiene validaciones, llamadas HTTP, SQL, mappers, reglas de negocio, navegación y manejo de permisos.

ViewModel = pantalla + negocio + datos + infraestructura

Esto viola separación de responsabilidades.

25.3 Repository anémico

El Repository no agrega abstracción real y solo reenvía llamadas sin criterio.

```
class UsuarioRepository(private val api: UsuarioApi) {  
    suspend fun get() = api.get()  
}
```

No siempre está mal, pero si nunca encapsula políticas de datos, cache, sincronización, mapeo o errores, hay que revisar si aporta valor.

25.4 Estado duplicado

```
var usuarios by remember { mutableStateOf(emptyList<Usuario>()) }  
val usuariosRoom = dao.obtenerUsuarios()  
val usuariosApi = api.obtenerUsuarios()
```

Múltiples fuentes simultáneas pueden producir inconsistencias.

25.5 Efectos secundarios sin control

```
@Composable  
fun Pantalla() {  
    viewModel.cargarDatos()  
}
```

Esto puede ejecutarse muchas veces por recomposición. Es mejor usar `LaunchedEffect` o iniciar la carga desde el `init` del `ViewModel`.

26 Tabla integradora de patrones

Patrón	Problema que resuelve
Composición de UI	Evita pantallas monolíticas y permite reutilizar componentes.
Screen - Content	Separa conexión arquitectónica de renderizado puro.
State hoisting	Permite controlar el estado desde un nivel superior.
UDF	Hace predecible el flujo de estado y eventos.
UI State	Agrupar el estado completo de una pantalla.
ViewModel	Conserva estado de UI y procesa eventos.
Repository	Abstrae el origen de datos.
DataSource	Encapsula una fuente concreta de datos.
Use Case	Encapsula reglas de aplicación o negocio.
DI / Hilt	Reduce acoplamiento y facilita pruebas.
Single Source of Truth	Evita inconsistencias entre copias de datos.
Offline First	Permite funcionamiento sin conectividad.
Side Effects	Controla operaciones fuera de la composición pura.

27 Arquitectura recomendada para proyectos iniciales

Para una app académica o profesional simple, una arquitectura razonable sería:

```
ui/  
  screens/  
  components/  
  viewmodel/  
domain/  
  model/  
  usecase/  
data/  
  repository/  
  local/  
  remote/  
  mapper/  
di/
```

Flujo recomendado:

```
Composable  
  -> ViewModel  
    -> UseCase  
      -> Repository  
        -> DataSource local/remoto
```

Para proyectos pequeños, puede simplificarse:

Composable -> ViewModel -> Repository

Pero conviene que los estudiantes entiendan que simplificar no significa mezclar responsabilidades.

28 A modo de cierre

Los patrones utilizados en Android con Jetpack Compose no son una colección de nombres aislados. Forman un sistema coherente alrededor de una idea principal: la UI es una función del estado, y los eventos del usuario deben modificar ese estado a través de caminos controlados.

Síntesis final

Compose obliga a pensar en estado, recomposición, flujo de datos y separación de responsabilidades. MVVM, UDF, State Hoisting, Repository, Use Case, DataSource e inyección de dependencias son respuestas complementarias al mismo problema: construir aplicaciones mantenibles, testeables y escalables.

Desde una perspectiva de ingeniería, dominar estos patrones permite pasar de una app que simplemente funciona a una app que puede evolucionar sin volverse frágil. La arquitectura no reemplaza al código: le da forma, límites y dirección.

29 Referencias sugeridas

- Android Developers. *Guide to app architecture*. <https://developer.android.com/topic/architecture>
- Android Developers. *Compose UI Architecture*. <https://developer.android.com/develop/ui/compose/architecture>
- Android Developers. *State and Jetpack Compose*. <https://developer.android.com/develop/ui/compose/state>
- Android Developers. *Where to hoist state*. <https://developer.android.com/develop/ui/compose/state-hoisting>
- Android Developers. *Side-effects in Compose*. <https://developer.android.com/develop/ui/compose/side-effects>
- Android Developers. *ViewModel overview*. <https://developer.android.com/topic/libraries/architecture/viewmodel>
- Android Developers. *Compose and other libraries*. <https://developer.android.com/develop/ui/compose/libraries>